4. Even if you do not write an `equals` method for a class, Java provides one. Describe the behavior of the `equals` method that Java automatically provides.
5. A "has a" relationship can exist between classes. What does this mean?
6. What happens if you attempt to call a method using a reference variable that is set to `null`?
7. Is it advisable or not advisable to write a method that returns a reference to an object that is a private field? What is the exception to this?
8. What is the `this` key word?
9. Look at the following declaration:

   ```
   enum Color { RED, ORANGE, GREEN, BLUE }
   ```

   a. What is the name of the data type declared by this statement?
   b. What are the `enum` constants for this type?
   c. Write a statement that defines a variable of this type and initializes it with a valid value.
10. Assuming the following `enum` declaration exists:

    ```
    enum Dog { POODLE, BOXER, TERRIER }
    ```

    what will the following statements display?

    a.
    ```
    System.out.println(Dog.POODLE + "\n" +
                       Dog.BOXER + "\n" +
                       Dog.TERRIER);
    ```

    b.
    ```
    System.out.println(Dog.POODLE.ordinal() + "\n" +
                       Dog.BOXER.ordinal() + "\n" +
                       Dog.TERRIER.ordinal());
    ```

    c.
    ```
    Dog myDog = Dog.BOXER;
    if (myDog.compareTo(Dog.TERRIER) > 0)
        System.out.println(myDog + " is greater than " +
                           Dog.TERRIER);
    else
       System.out.println(myDog + " is NOT greater than " +
                          Dog.TERRIER);
    ```
11. Under what circumstances does an object become a candidate for garbage collection?

Programming Challenges

MyProgrammingLab™ *Visit www.myprogramminglab.com to complete many of these Programming Challenges online and get instant feedback.*

1. `Area` Class

Write a class that has three overloaded static methods for calculating the areas of the following geometric shapes:

- circles
- rectangles
- cylinders

Here are the formulas for calculating the area of the shapes.

Area of a circle:	$Area = \pi r^2$
where	π is `Math.PI` and r is the circle's radius
Area of a rectangle:	$Area = Width \times Length$
Area of a cylinder:	$Area = \pi r^2 h$
where	π is `Math.PI`, r is the radius of the cylinder's base, and h is the cylinder's height

Because the three methods are to be overloaded, they should each have the same name, but different parameter lists. Demonstrate the class in a complete program.

VideoNote

The `BankAccount` Class Copy Constructor Problem

2. `BankAccount` Class Copy Constructor

Add a copy constructor to the `BankAccount` class. This constructor should accept a `BankAccount` object as an argument. It should assign to the `balance` field the value in the argument's `balance` field. As a result, the new object will be a copy of the argument object.

3. Carpet Calculator

The Westfield Carpet Company has asked you to write an application that calculates the price of carpeting for rectangular rooms. To calculate the price, you multiply the area of the floor (width times length) by the price per square foot of carpet. For example, the area of floor that is 12 feet long and 10 feet wide is 120 square feet. To cover that floor with carpet that costs $8 per square foot would cost $960. ($12 \times 10 \times 8 = 960$.)

First, you should create a class named `RoomDimension` that has two fields: one for the length of the room and one for the width. The `RoomDimension` class should have a method that returns the area of the room. (The area of the room is the room's length multiplied by the room's width.)

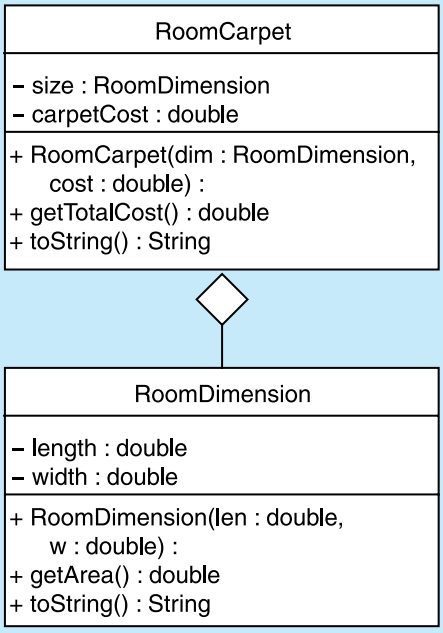
Next you should create a `RoomCarpet` class that has a `RoomDimension` object as a field. It should also have a field for the cost of the carpet per square foot. The `RoomCarpet` class should have a method that returns the total cost of the carpet.

Figure 8-21 is a UML diagram that shows possible class designs and the relationships among the classes. Once you have written these classes, use them in an application that asks the user to enter the dimensions of a room and the price per square foot of the desired carpeting. The application should display the total cost of the carpet.

4. `LandTract` Class

Make a `LandTract` class that has two fields: one for the tract's length and one for the width. The class should have a method that returns the tract's area, as well as an `equals` method and a `toString` method. Demonstrate the class in a program that asks the user to enter the dimensions for two tracts of land. The program should display the area of each tract of land and indicate whether the tracts are of equal size.

Figure 8-21 UML diagram for Programming Challenge 3



5. `Month` Class

Write a class named `Month`. The class should have an `int` field named `monthNumber` that holds the number of the month. For example, January would be 1, February would be 2, and so forth. In addition, provide the following methods:

- A no-arg constructor that sets the `monthNumber` field to 1.
- A constructor that accepts the number of the month as an argument. It should set the `monthNumber` field to the value passed as the argument. If a value less than 1 or greater than 12 is passed, the constructor should set `monthNumber` to 1.
- A constructor that accepts the name of the month, such as "January" or "February" as an argument. It should set the `monthNumber` field to the correct corresponding value.
- A `setMonthNumber` method that accepts an `int` argument, which is assigned to the `monthNumber` field. If a value less than 1 or greater than 12 is passed, the method should set `monthNumber` to 1.
- A `getMonthNumber` method that returns the value in the `monthNumber` field.
- A `getMonthName` method that returns the name of the month. For example, if the `monthNumber` field contains 1, then this method should return "January".
- A `toString` method that returns the same value as the `getMonthName` method.
- An `equals` method that accepts a `Month` object as an argument. If the argument object holds the same data as the calling object, this method should return `true`. Otherwise, it should return `false`.

- A `greaterThan` method that accepts a `Month` object as an argument. If the calling object's `monthNumber` field is greater than the argument's `monthNumber` field, this method should return `true`. Otherwise, it should return `false`.
- A `lessThan` method that accepts a `Month` object as an argument. If the calling object's `monthNumber` field is less than the argument's `monthNumber` field, this method should return `true`. Otherwise, it should return `false`.

6. `CashRegister` Class

Write a `CashRegister` class that can be used with the `RetailItem` class that you wrote in Chapter 6's Programming Challenge 4. The `CashRegister` class should simulate the sale of a retail item. It should have a constructor that accepts a `RetailItem` object as an argument. The constructor should also accept an integer that represents the quantity of items being purchased. In addition, the class should have the following methods:

- The `getSubtotal` method should return the subtotal of the sale, which is the quantity multiplied by the price. This method must get the price from the `RetailItem` object that was passed as an argument to the constructor.
- The `getTax` method should return the amount of sales tax on the purchase. The sales tax rate is 6 percent of a retail sale.
- The `getTotal` method should return the total of the sale, which is the subtotal plus the sales tax.

Demonstrate the class in a program that asks the user for the quantity of items being purchased, and then displays the sale's subtotal, amount of sales tax, and total.

7. Sales Receipt File

Modify the program you wrote in Programming Challenge 6 to create a file containing a sales receipt. The program should ask the user for the quantity of items being purchased, and then generate a file with contents similar to the following:

```
SALES RECEIPT
Unit Price: $10.00
Quantity: 5
Subtotal: $50.00
Sales Tax: $ 3.00
Total: $53.00
```

8. Parking Ticket Simulator

For this assignment you will design a set of classes that work together to simulate a police officer issuing a parking ticket. You should design the following classes:

- **The `ParkedCar` Class:** This class should simulate a parked car. The class's responsibilities are as follows:
  - To know the car's make, model, color, license number, and the number of minutes that the car has been parked.
- **The `ParkingMeter` Class:** This class should simulate a parking meter. The class's only responsibility is as follows:
  - To know the number of minutes of parking time that has been purchased.

- **The `ParkingTicket` Class:** This class should simulate a parking ticket. The class's responsibilities are as follows:
  - To report the make, model, color, and license number of the illegally parked car
  - To report the amount of the fine, which is $25 for the first hour or part of an hour that the car is illegally parked, plus $10 for every additional hour or part of an hour that the car is illegally parked
  - To report the name and badge number of the police officer issuing the ticket
- **The `PoliceOfficer` Class:** This class should simulate a police officer inspecting parked cars. The class's responsibilities are as follows:
  - To know the police officer's name and badge number
  - To examine a `ParkedCar` object and a `ParkingMeter` object, and determine whether the car's time has expired
  - To issue a parking ticket (generate a `ParkingTicket` object) if the car's time has expired

Write a program that demonstrates how these classes collaborate.

9. Geometry Calculator

Design a `Geometry` class with the following methods:

- A static method that accepts the radius of a circle and returns the area of the circle. Use the following formula:
  $Area = \pi r^2$
  Use `Math.PI` for π and the radius of the circle for r.
- A static method that accepts the length and width of a rectangle and returns the area of the rectangle. Use the following formula:
  $Area = Length \times Width$
- A static method that accepts the length of a triangle's base and the triangle's height. The method should return the area of the triangle. Use the following formula:
  $Area = Base \times Height \times 0.5$

The methods should display an error message if negative values are used for the circle's radius, the rectangle's length or width, or the triangle's base or height.

Next, write a program to test the class, which displays the following menu and responds to the user's selection:

```
Geometry Calculator
1. Calculate the Area of a Circle
2. Calculate the Area of a Rectangle
3. Calculate the Area of a Triangle
4. Quit

Enter your choice (1-4):
```

Display an error message if the user enters a number outside the range of 1 through 4 when selecting an item from the menu.

10. Car Instrument Simulator

For this assignment, you will design a set of classes that work together to simulate a car's fuel gauge and odometer. The classes you will design are the following:

- **The `FuelGauge` Class:** This class will simulate a fuel gauge. Its responsibilities are as follows:
  - To know the car's current amount of fuel, in gallons.
  - To report the car's current amount of fuel, in gallons.
  - To be able to increment the amount of fuel by 1 gallon. This simulates putting fuel in the car. (The car can hold a maximum of 15 gallons.)
  - To be able to decrement the amount of fuel by 1 gallon, if the amount of fuel is greater than 0 gallons. This simulates burning fuel as the car runs.
- **The `Odometer` Class:** This class will simulate the car's odometer. Its responsibilities are as follows:
  - To know the car's current mileage.
  - To report the car's current mileage.
  - To be able to increment the current mileage by 1 mile. The maximum mileage the odometer can store is 999,999 miles. When this amount is exceeded, the odometer resets the current mileage to 0.
  - To be able to work with a `FuelGauge` object. It should decrease the `FuelGauge` object's current amount of fuel by 1 gallon for every 24 miles traveled. (The car's fuel economy is 24 miles per gallon.)

Demonstrate the classes by creating instances of each. Simulate filling the car up with fuel, and then run a loop that increments the odometer until the car runs out of fuel. During each loop iteration, print the car's current mileage and amount of fuel.

11. First to One Game

This game is meant for two or more players. In the game, each player starts out with 50 points, as each player takes a turn rolling the dice; the amount generated by the dice is subtracted from the player's points. The first player with exactly one point remaining wins. If a player's remaining points minus the amount generated by the dice results in a value less than one, then the amount should be added to the player's points. (As an alternative, the game can be played with a set number turns. In this case, the player with the amount of points closest to one, when all rounds have been played, wins.)

Write a program that simulates the game being played by two players. Use the `Die` class that was presented in Chapter 6 to simulate the dice. Write a `Player` class to simulate the players.

12. Heads or Tails Game

This game is meant for two or more players. In this game, the players take turns flipping a coin. Before the coin is flipped, players should guess if the coin will land face up or face down. If a player guesses correctly, then that player is awarded a point. If a player guesses incorrectly, then that player will lose a point. The first player to score five points is the winner.

Write a program that simulates the game being played by two players. Use the `Coin` class that you wrote as an assignment in Chapter 6 (Programming Challenge 16) to simulate the coin. Write a `Player` class to simulate the players.